

[Open in app](#)[Sign up](#)[Sign in](#)

Medium



The JavaScript Event Loop Explained with Examples

5 min read · Dec 16, 2024



Frontend Highlights

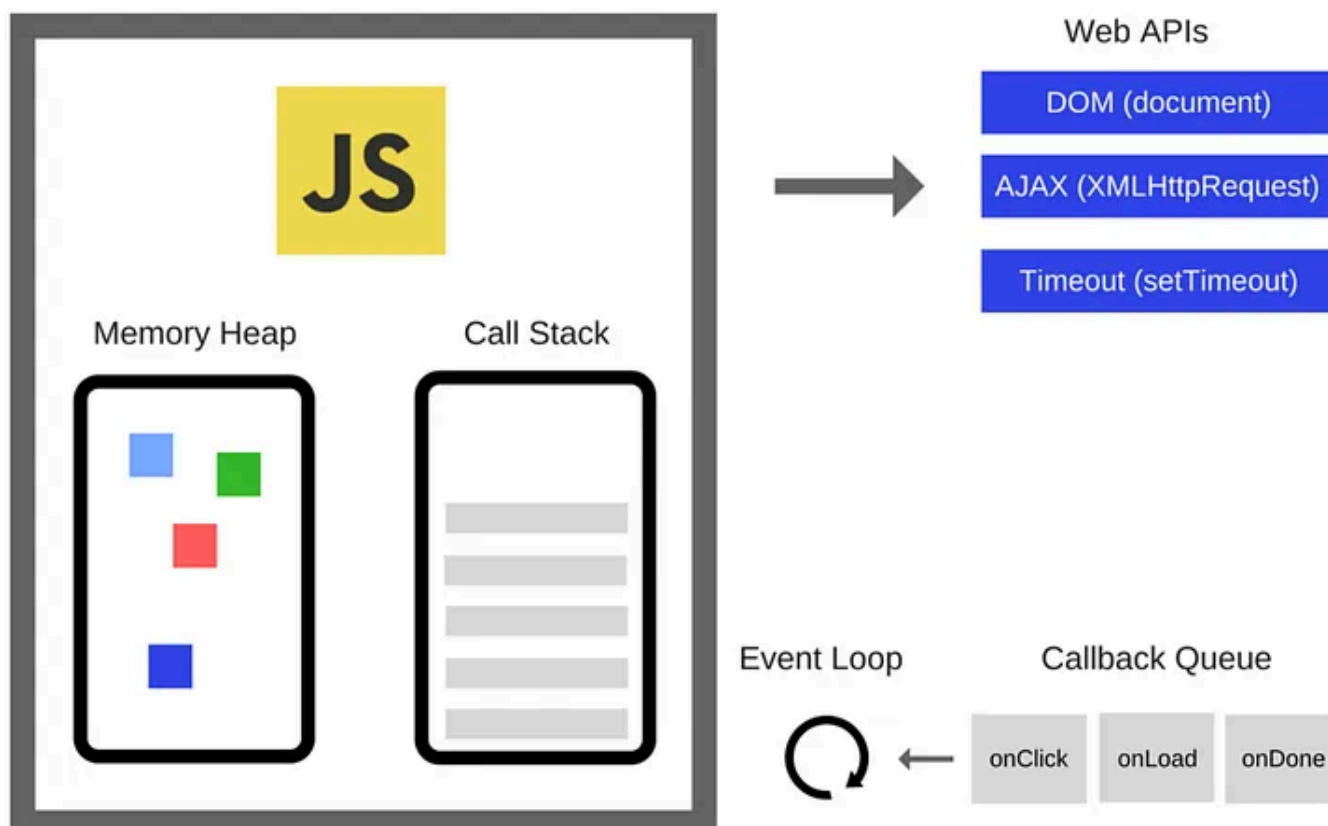
[Follow](#)

Listen



Share

The event loop is a core concept in JavaScript that enables non-blocking, asynchronous behavior. Understanding how the event loop works is essential to mastering JavaScript, especially for building responsive web applications. This article dives into the event loop, explains the different types of tasks in JavaScript, and provides ten examples to solidify your understanding.



What is the Event Loop?

At its core, the JavaScript event loop is responsible for managing the execution of code, collecting and processing events, and executing queued tasks. JavaScript operates in a single-threaded environment, meaning only one piece of code runs at a time. The event loop ensures that tasks are executed in the correct order, enabling asynchronous programming.

To explain it in simple terms, imagine you're running a to-do list. The event loop is like a manager that ensures tasks are completed in the right order, depending on their type and urgency.

Components of the Event Loop

1. **Call Stack:** Keeps track of function calls. When a function is invoked, it is pushed onto the stack. When the function finishes execution, it is popped off.
2. **Web APIs:** Provides browser features like `setTimeout`, DOM events, and HTTP requests. These APIs handle asynchronous operations.
3. **Task Queue (Callback Queue):** Stores tasks waiting to be executed after the call stack is empty. These tasks are queued by `setTimeout`, `setInterval`, or other APIs.
4. **Microtask Queue:** A higher-priority queue for promises and `MutationObserver` callbacks. Microtasks are executed before tasks in the task queue.
5. **Event Loop:** Continuously checks if the call stack is empty and pushes tasks from the microtask queue or task queue to the call stack for execution.

How It Works (Simplified with an analogy):

1. **Your Main Task:** JavaScript executes code line by line in a single thread (like following a recipe). This is called the **call stack**.
2. **Waiting Tasks (Events):** Some tasks take time (e.g., fetching data from the internet, timers). Instead of blocking progress, these tasks are sent to “wait in line” in a queue (known as the **event queue**).
3. **The Manager (Event Loop):** The event loop constantly checks:
 - Is the main task (call stack) empty?
 - Are there any tasks waiting in the queue?If yes, it picks a task from the queue and moves it to the stack for execution.

Event Loop Analogy

Imagine you're at a restaurant:

1. **The Chef (Call Stack):** The chef prepares one order at a time. If a dish takes a long time to cook, the chef moves it to a separate station (like the oven or timer) and starts working on the next order.
2. **The Waiter (Event Queue):** The waiter keeps an eye on all pending tasks (like dishes in the oven) and brings them back to the chef once they're ready.
3. **The Manager (Event Loop):** The manager ensures the chef only works on one task at a time and keeps the workflow smooth.

. . .

Types of Tasks in JavaScript

1. **Synchronous Tasks:** Executed immediately on the call stack (e.g., function calls, variable declarations).
2. **Microtasks:** High-priority asynchronous tasks, such as `Promise` callbacks and `queueMicrotask`.
3. **Macrotasks:** Lower-priority asynchronous tasks, like `setTimeout`, `setInterval`, and DOM events.

Order of Execution

1. Execute all synchronous tasks on the call stack.
2. Process all microtasks in the microtask queue.
3. Process the first task in the macrotask queue.
4. Repeat.

. . .

Examples

You can run all examples [JavaScript Visualizer 9000](https://medium.com/@ignatovich.dm/the-javascript-event-loop-explained-with-examples-d8f7ddf0861d)

Example 1: Basic Synchronous and Asynchronous Code

```
console.log('A'); // Synchronous

setTimeout(() => {
  console.log('B'); // Macrotask
}, 0);

console.log('C'); // Synchronous
```

Output: A, C, B

Example 2: Microtasks with Promises

```
console.log('A');

setTimeout(() => {
  console.log('B'); // Macrotask
}, 0);

Promise.resolve().then(() => {
  console.log('C'); // Microtask
});

console.log('D');
```

Output: A, D, C, B

Example 3: Nested Microtasks

```
Promise.resolve().then(() => {
  console.log('A');
  Promise.resolve().then(() => console.log('B'));
});

console.log('C');
```

Output: C, A, B

Example 4: setTimeout vs setImmediate (Node.js only)

```
setTimeout(() => console.log('A'), 0); // Macrotask
setImmediate(() => console.log('B')); // Macrotask in Node.js
```

Output (depends on Node.js version): Generally A, then B.

Example 5: Event Listener and Promises

```
document.body.addEventListener('click', () => {
  console.log('Click Event'); // Macrotask
});

Promise.resolve().then(() => console.log('Promise Resolved')); // Microtask
console.log('End');
```

Output: End, Promise Resolved, Click Event

Example 6: Interleaving Promises and setTimeout

```
setTimeout(() => console.log('A'), 0);
Promise.resolve().then(() => console.log('B'));
setTimeout(() => console.log('C'), 0);
Promise.resolve().then(() => console.log('D'));
```

Output: B, D, A, C

Get Frontend Highlights's stories in your inbox

Join Medium for free to get updates from this writer.

Subscribe

☒ Remember me for faster sign in

Example 7: Nested Promises with `setTimeout`

```
console.log('A');

setTimeout(() => {
  console.log('B');
  Promise.resolve().then(() => {
    console.log('C');
  });
}, 0);

Promise.resolve().then(() => {
  console.log('D');
  setTimeout(() => {
    console.log('E');
  }, 0);
});

console.log('F');
```

Output:

1. A (Synchronous)
2. F (Synchronous)
3. D (Microtask from `Promise`)
4. B (Macrotask from `setTimeout`)
5. C (Microtask created within the `setTimeout`)
6. E (Macrotask from the inner `setTimeout`)

Example 8: Promise Chaining with `setTimeout`

```
console.log('1');

setTimeout(() => {
  console.log('2');
  Promise.resolve().then(() => {
    console.log('3');
  }).then(() => {
```

```
    console.log('4');
  });
}, 0);

Promise.resolve().then(() => {
  console.log('5');
}).then(() => {
  console.log('6');
});

console.log('7');
```

Output:

1. 1 (Synchronous)
2. 7 (Synchronous)
3. 5 (Microtask from first Promise)
4. 6 (Chained Microtask)
5. 2 (Macrotask from setTimeout)
6. 3 (Microtask from Promise inside setTimeout)
7. 4 (Chained Microtask)

Example 9: Mixing Promise Resolution with Delays

```
console.log('Start');

setTimeout(() => {
  console.log('Timeout 1');
}, 0);

Promise.resolve().then(() => {
  console.log('Promise 1');
  setTimeout(() => {
    console.log('Timeout 2');
  }, 0);
  return Promise.resolve();
}).then(() => {
  console.log('Promise 2');
});
```

```
console.log('End');
```

Output:

1. Start (Synchronous)
2. End (Synchronous)
3. Promise 1 (Microtask)
4. Promise 2 (Chained Microtask)
5. Timeout 1 (Macrotask from first `setTimeout`)
6. Timeout 2 (Macrotask from `setTimeout` inside Promise)

Example 10: Deeply Nested Promises in a Timer

```
setTimeout(() => {  
  console.log('Timer 1');  
  Promise.resolve().then(() => {  
    console.log('Microtask 1');  
    Promise.resolve().then(() => {  
      console.log('Microtask 2');  
    });  
  });  
}, 0);  
  
Promise.resolve().then(() => {  
  console.log('Microtask 3');  
});  
  
console.log('Main Task');
```

Output:

1. Main Task (Synchronous)
2. Microtask 3 (Microtask from Promise)
3. Timer 1 (Macrotask from `setTimeout`)

4. **Microtask 1** (Microtask within `setTimeout`)
5. **Microtask 2** (Chained Microtask from `Microtask 1`)

Example 11: Combining Chained Promises and Timer Nesting

```
console.log('Start');

setTimeout(() => {
  console.log('Timeout 1');
  Promise.resolve().then(() => {
    console.log('Promise 1');
  }).then(() => {
    console.log('Promise 2');
  });
}, 0);

Promise.resolve().then(() => {
  console.log('Promise 3');
  setTimeout(() => {
    console.log('Timeout 2');
  }, 0);
  return Promise.resolve();
}).then(() => {
  console.log('Promise 4');
});

console.log('End');
```

Output:

1. **Start** (Synchronous)
2. **End** (Synchronous)
3. **Promise 3** (Microtask from first `Promise`)
4. **Promise 4** (Chained Microtask)
5. **Timeout 1** (Macrotask from first `setTimeout`)
6. **Promise 1** (Microtask within `Timeout 1`)
7. **Promise 2** (Chained Microtask within `Timeout 1`)

8. Timeout 2 (Macrotask from `setTimeout` inside `Promise`)

Key points

1. **Microtasks (Promises)** always execute before macrotasks (`setTimeout`, `setInterval`, etc.).
2. Nesting Promises creates a queue of microtasks that execute in the same cycle.
3. Timers (`setTimeout`) will always defer execution to the next event loop cycle, while microtasks resolve immediately after the current task.

Js

Event Loop

JavaScript

Javascript Development



Follow

Written by Frontend Highlights

4.8K followers · 7 following

This page is about simple frontend things which can be useful in real work Support us on buymeacoffee.com/guestdm or <https://ko-fi.com/dm110416>



Frontend Highlights

Frontend Highlights is a collection of articles about frontend development, including JavaScript, CSS, and HTML. The articles are written by Frontend Highlights, a frontend developer and content creator. The articles are aimed at helping developers learn and improve their skills. The articles are written in a simple and easy-to-understand way. The articles are also written in a way that is helpful and practical. The articles are also written in a way that is fun and engaging. The articles are also written in a way that is informative and educational. The articles are also written in a way that is entertaining and enjoyable. The articles are also written in a way that is inspiring and motivating. The articles are also written in a way that is empowering and uplifting. The articles are also written in a way that is encouraging and supportive. The articles are also written in a way that is helpful and useful. The articles are also written in a way that is informative and educational. The articles are also written in a way that is entertaining and enjoyable. The articles are also written in a way that is inspiring and motivating. The articles are also written in a way that is empowering and uplifting. The articles are also written in a way that is encouraging and supportive. The articles are also written in a way that is helpful and useful.

Frontend Highlights is a collection of articles about frontend development, including JavaScript, CSS, and HTML. The articles are written by Frontend Highlights, a frontend developer and content creator. The articles are aimed at helping developers learn and improve their skills. The articles are written in a simple and easy-to-understand way. The articles are also written in a way that is helpful and practical. The articles are also written in a way that is fun and engaging. The articles are also written in a way that is informative and educational. The articles are also written in a way that is entertaining and enjoyable. The articles are also written in a way that is inspiring and motivating. The articles are also written in a way that is empowering and uplifting. The articles are also written in a way that is encouraging and supportive. The articles are also written in a way that is helpful and useful.

